



Chapter 13. *Explicit Locks*

Before Java 5.0, the only mechanisms for coordinating access to shared data were `synchronized` and `volatile`. Java 5.0 adds another option: `ReentrantLock`. Contrary to what some have written, `ReentrantLock` is not a replacement for intrinsic locking, but rather an alternative with advanced features for when intrinsic locking proves too limited.

13.1. Lock and ReentrantLock

The `Lock` interface, shown in [Listing 13.1](#), defines a number of abstract locking operations. Unlike intrinsic locking, `Lock` offers a choice of unconditional, polled, timed, and interruptible lock acquisition, and all lock and unlock operations are explicit. `Lock` implementations must provide the same memory-visibility semantics as intrinsic locks, but can differ in their locking semantics, scheduling algorithms, ordering guarantees, and performance characteristics. (`Lock.newCondition` is covered in [Chapter 14](#).)

Listing 13.1. Lock Interface.

```
public interface Lock {  
    void lock();  
    void lockInterruptibly() throws InterruptedException;  
    boolean tryLock();  
    boolean tryLock(long timeout, TimeUnit unit)  
        throws InterruptedException;  
    void unlock();  
    Condition newCondition();  
}
```

`ReentrantLock` implements `Lock`, providing the same mutual exclusion and memory-visibility guarantees as `synchronized`. Acquiring a `ReentrantLock` has the same memory semantics as entering a `synchronized` block, and releasing a `ReentrantLock` has the same memory semantics as exiting a `synchronized` block. (Memory visibility is cov-

ered in [Section 3.1](#) and in [Chapter 16](#).) And, like `synchronized`, `ReentrantLock` offers reentrant locking semantics (see [Section 2.3.2](#)). `ReentrantLock` supports all of the lock-acquisition modes defined by `Lock`, providing more flexibility for dealing with lock unavailability than does `synchronized`.

Why create a new locking mechanism that is so similar to intrinsic locking? Intrinsic locking works fine in most situations but has some functional limitations—it is not possible to interrupt a thread waiting to acquire a lock, or to attempt to acquire a lock without being willing to wait for it forever. Intrinsic locks also must be released in the same block of code in which they are acquired; this simplifies coding and interacts nicely with exception handling, but makes non-blockstructured locking disciplines impossible. None of these are reasons to abandon `synchronized`, but in some cases a more flexible locking mechanism offers better liveness or performance.

[Listing 13.2](#) shows the canonical form for using a `Lock`. This idiom is somewhat more complicated than using intrinsic locks: the lock *must* be released in a `finally` block. Otherwise, the lock would never be released if the guarded code were to throw an exception. When using locking, you must also consider what happens if an exception is thrown out of the `try` block; if it is possible for the object to be left in an inconsistent state, additional `try-catch` or `try-finally` blocks may be needed. (You should always consider the effect of exceptions when using any form of locking, including intrinsic locking.)

Failing to use `finally` to release a `Lock` is a ticking time bomb. When it goes off, you will have a hard time tracking down its origin as there will be no record of where or when the `Lock` should have been released. This is one reason not to use `ReentrantLock` as a blanket substitute for `synchronized`: it is more “dangerous” because it doesn't automatically clean up the lock when control leaves the guarded block. While remembering to release the lock from a `finally` block is not all that difficult, it is also not impossible to forget. [\[1\]](#)

[\[1\]](#) FindBugs has an “unreleased lock” detector identifying when a `Lock` is not released in all code paths out of the block in which it was acquired.

Listing 13.2. Guarding Object State Using `ReentrantLock`.

```
Lock lock = new ReentrantLock();
...
lock.lock();
try {
    // update object state
    // catch exceptions and restore invariants if necessary
} finally {
    lock.unlock();
}
```

13.1.1. Polled and Timed Lock Acquisition

The timed and polled lock-acquisition modes provided by `tryLock` allow more sophisticated error recovery than unconditional acquisition. With intrinsic locks, a deadlock is fatal—the only way to recover is to restart the application, and the only defense is to construct your program so that inconsistent lock ordering is impossible. Timed and polled locking offer another option: probabilistic deadlock avoidance.

Using timed or polled lock acquisition (`tryLock`) lets you regain control if you cannot acquire all the required locks, release the ones you did acquire, and try again (or at least log the failure and do something else).

Listing 13.3 shows an alternate way of addressing the dynamic ordering deadlock from **Section 10.1.2**: use `tryLock` to attempt to acquire both locks, but back off and retry if they cannot both be acquired. The sleep time has a fixed component and a random component to reduce the likelihood of livelock. If the locks cannot be acquired within the specified time, `transferMoney` returns a failure status so that the operation can fail gracefully. (See [CPJ 2.5.1.2] and [CPJ 2.5.1.3] for more examples of using polled locks for deadlock avoidance.)

Timed locks are also useful in implementing activities that manage a time budget (see **Section 6.3.7**). When an activity with a time budget calls a blocking method, it can supply a timeout corresponding to the remaining time in the budget. This lets activities terminate early if they cannot deliver a result within the desired time. With intrinsic locks, there is no way to cancel a lock acquisition once it is started, so intrinsic locks put the ability to implement time-budgeted activities at risk.

The travel portal example in [Listing 6.17](#) on page [134](#) creates a separate task for each car-rental company from which it was soliciting bids. Soliciting a bid probably involves some sort of network-based request mechanism, such as a web service request. But soliciting a bid might also require exclusive access to a scarce resource, such as a direct communications line to the company.

We saw one way to ensure serialized access to a resource in [Section 9.5](#): a single-threaded executor. Another approach is to use an exclusive lock to guard access to the resource. The code in [Listing 13.4](#) tries to send a message on a shared communications line guarded by a `Lock`, but fails gracefully if it cannot do so within its time budget. The timed `tryLock` makes it practical to incorporate exclusive locking into such a time-limited activity.

13.1.2. Interruptible Lock Acquisition

Just as timed lock acquisition allows exclusive locking to be used within timelimited activities, interruptible lock acquisition allows locking to be used within cancellable activities. [Section 7.1.6](#) identified several mechanisms, such as acquiring an intrinsic lock, that are not responsive to interruption. These noninterruptible blocking mechanisms complicate the implementation of cancellable tasks. The `lockInterruptibly` method allows you to try to acquire a lock while remaining responsive to interruption, and its inclusion in `Lock` avoids creating another category of non-interruptible blocking mechanisms.

Listing 13.3. Avoiding Lock-ordering Deadlock Using `Trylock`.

```

public boolean transferMoney(Account fromAcct,
                             Account toAcct,
                             DollarAmount amount,
                             long timeout,
                             TimeUnit unit)
    throws InsufficientFundsException, InterruptedException {
    long fixedDelay = getFixedDelayComponentNanos(timeout, unit);
    long randMod = getRandomDelayModulusNanos(timeout, unit);
    long stopTime = System.nanoTime() + unit.toNanos(timeout);

    while (true) {
        if (fromAcct.lock.tryLock()) {
            try {
                if (toAcct.lock.tryLock()) {
                    try {
                        if (fromAcct.getBalance().compareTo(amount)
                            < 0)
                            throw new InsufficientFundsException();
                        else {
                            fromAcct.debit(amount);
                            toAcct.credit(amount);
                            return true;
                        }
                    } finally {
                        toAcct.lock.unlock();
                    }
                }
            } finally {
                fromAcct.lock.unlock();
            }
        }
        if (System.nanoTime() > stopTime)
            return false;
        NANOSECONDS.sleep(fixedDelay + rnd.nextLong() % randMod);
    }
}

```

Listing 13.4. Locking with a Time Budget.

```

public boolean trySendOnSharedLine(String message,
                                    long timeout, TimeUnit unit)
    throws InterruptedException {
    long nanosToLock = unit.toNanos(timeout)
        - estimatedNanosToSend(message);
    if (!lock.tryLock(nanosToLock, NANOSECONDS))
        return false;
    try {
        return sendOnSharedLine(message);
    } finally {
        lock.unlock();
    }
}

```

The canonical structure of interruptible lock acquisition is slightly more complicated than normal lock acquisition, as two `try` blocks are needed. (If the interruptible lock acquisition can throw `InterruptedException`, the standard `try-finally` locking idiom works.) [Listing 13.5](#) uses `lockInterruptibly` to implement `sendOnSharedLine` from [Listing 13.4](#) so that we can call it from a cancellable task. The timed `tryLock` is also responsive to interruption and so can be used when you need both timed and interruptible lock acquisition.

Listing 13.5. Interruptible Lock Acquisition.

```
public boolean sendOnSharedLine(String message)
    throws InterruptedException {
    lock.lockInterruptibly();
    try {
        return cancellableSendOnSharedLine(message);
    } finally {
        lock.unlock();
    }
}

private boolean cancellableSendOnSharedLine(String message)
    throws InterruptedException { ... }
```

13.1.3. Non-block-structured Locking

With intrinsic locks, acquire-release pairs are block-structured—a lock is always released in the same basic block in which it was acquired, regardless of how control exits the block. Automatic lock release simplifies analysis and prevents potential coding errors, but sometimes a more flexible locking discipline is needed.

In [Chapter 11](#), we saw how reducing lock granularity can enhance scalability. Lock striping allows different hash chains in a hash-based collection to use different locks. We can apply a similar principle to reduce locking granularity in a linked list by using a separate lock for *each link node*, allowing different threads to operate independently on different portions of the list. The lock for a given node guards the link pointers and the data stored in that node, so when traversing or modifying the list we must hold the lock on one node until we acquire the lock on the next node; only then can we release the lock on the first node. An example of

this technique, called *hand-over-hand locking* or *lock coupling*, appears in [CPJ 2.5.1.4].

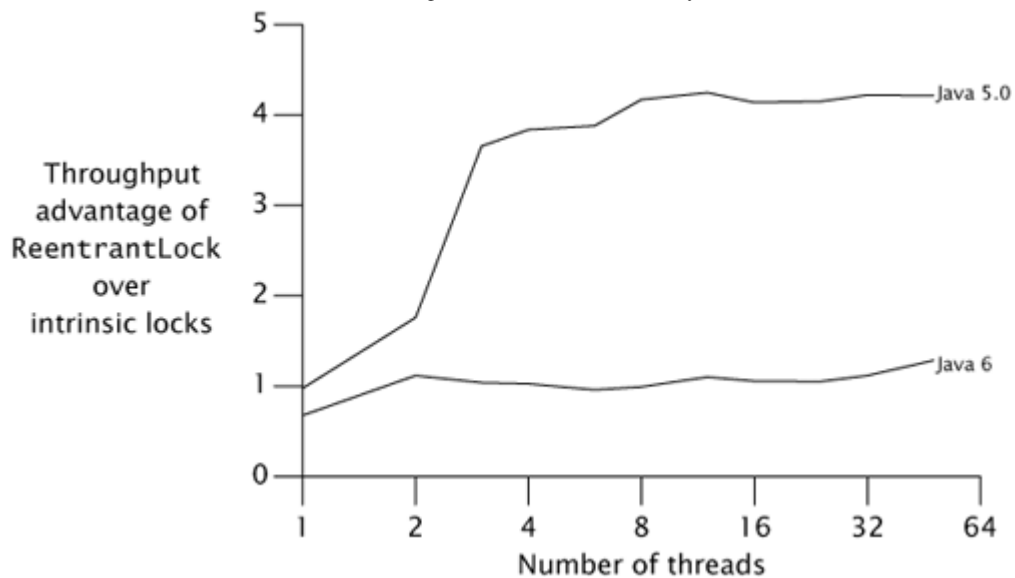
13.2. Performance Considerations

When `ReentrantLock` was added in Java 5.0, it offered far better contended performance than intrinsic locking. For synchronization primitives, contended performance is the key to scalability: if more resources are expended on lock management and scheduling, fewer are available for the application. A better lock implementation makes fewer system calls, forces fewer context switches, and initiates less memory-synchronization traffic on the shared memory bus, operations that are time-consuming and divert computing resources from the program.

Java 6 uses an improved algorithm for managing intrinsic locks, similar to that used by `ReentrantLock`, that closes the scalability gap considerably. [Figure 13.1](#) shows the performance difference between intrinsic locks and `ReentrantLock` on Java 5.0 and on a prerelease build of Java 6 on a four-way Opteron system running Solaris. The curves represent the “speedup” of `ReentrantLock` over intrinsic locking on a single JVM version. On Java 5.0, `ReentrantLock` offers considerably better throughput, but on Java 6, the two are quite close.^[2] The test program is the same one used in [Section 11.5](#), this time comparing the throughput of a `HashMap` guarded by an intrinsic lock and by a `ReentrantLock`.

^[2] Though this particular graph doesn't show it, the scalability difference between Java 5.0 and Java 6 really does come from improvement in intrinsic locking, rather than from regression in `ReentrantLock`.

Figure 13.1. Intrinsic Locking Versus `ReentrantLock` Performance on Java 5.0 and Java 6.



On Java 5.0, the performance of intrinsic locking drops dramatically in going from one thread (no contention) to more than one thread; the performance of `ReentrantLock` drops far less, showing its better scalability. But on Java 6, it is a different story—intrinsic locks no longer fall apart under contention, and the two scale fairly similarly.

Graphs like [Figure 13.1](#) remind us that statements of the form “X is faster than Y” are at best short-lived. Performance and scalability are sensitive to platform factors such as CPU, processor count, cache size, and JVM characteristics, all of which can change over time. [\[3\]](#)

[\[3\]](#) When we started this book, `ReentrantLock` seemed the last word in lock scalability. Less than a year later, intrinsic locking gives it a good run for its money. Performance is not just a moving target, it can be a fast-moving target.

Performance is a moving target; yesterday's benchmark showing that X is faster than Y may already be out of date today.

13.3. Fairness

The `ReentrantLock` constructor offers a choice of two *fairness* options: create a *nonfair* lock (the default) or a *fair* lock. Threads acquire a fair lock in the order in which they requested it, whereas a nonfair lock permits *barging*: threads requesting a lock can jump ahead of the queue of

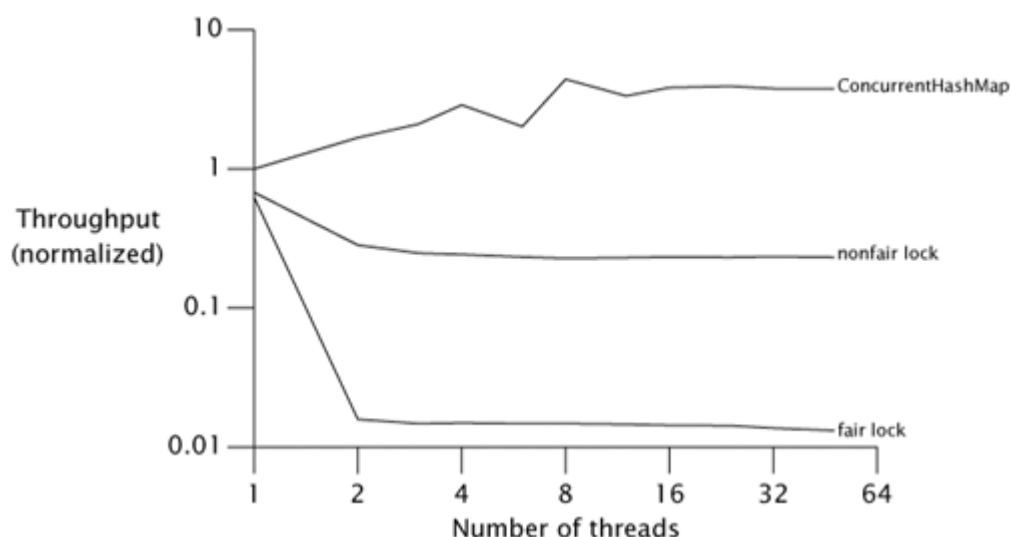
waiting threads if the lock happens to be available when it is requested. (`Semaphore` also offers the choice of fair or nonfair acquisition ordering.) Nonfair `ReentrantLock`s do not go out of their way to promote barging—they simply don't prevent a thread from barging if it shows up at the right time. With a fair lock, a newly requesting thread is queued if the lock is held by another thread or if threads are queued waiting for the lock; with a nonfair lock, the thread is queued only if the lock is currently held.^[4]

^[4] The polled `tryLock` always barges, even for fair locks.

Wouldn't we want all locks to be fair? After all, fairness is good and unfairness is bad, right? (Just ask your kids.) When it comes to locking, though, fairness has a significant performance cost because of the overhead of suspending and resuming threads. In practice, a statistical fairness guarantee—promising that a blocked thread will *eventually* acquire the lock—is often good enough, and is far less expensive to deliver. Some algorithms rely on fair queueing to ensure their correctness, but these are unusual. In most cases, the performance benefits of nonfair locks outweigh the benefits of fair queueing.

Figure 13.2 shows another run of the `Map` performance test, this time comparing `HashMap` wrapped with fair and nonfair `ReentrantLock`s on a four-way Opteron system running Solaris, plotted on a log scale.^[5] The fairness penalty is nearly two orders of magnitude. *Don't pay for fairness if you don't need it.*

^[5] The graph for `ConcurrentHashMap` is fairly wiggly in the region between four and eight threads. These variations almost certainly come from measurement noise, which could be introduced by coincidental interactions with the hash codes of the elements, thread scheduling, map resizing, garbage collection or other memory-system effects, or by the OS deciding to run some periodic housekeeping task around the time that test case ran. The reality is that there are all sorts of variations in performance tests that usually aren't worth bothering to control. We made no attempt to clean up our graphs artificially, because real-world performance measurements are also full of noise.

Figure 13.2. Fair Versus Nonfair Lock Performance.

One reason barging locks perform so much better than fair locks under heavy contention is that there can be a significant delay between when a suspended thread is resumed and when it actually runs. Let's say thread *A* holds a lock and thread *B* asks for that lock. Since the lock is busy, *B* is suspended. When *A* releases the lock, *B* is resumed so it can try again. In the meantime, though, if thread *C* requests the lock, there is a good chance that *C* can acquire the lock, use it, and release it before *B* even finishes waking up. In this case, everyone wins: *B* gets the lock no later than it otherwise would have, *C* gets it much earlier, and throughput is improved.

Fair locks tend to work best when they are held for a relatively long time or when the mean time between lock requests is relatively long. In these cases, the condition under which barging provides a throughput advantage—when the lock is unheld but a thread is currently waking up to claim it—is less likely to hold.

Like the default `ReentrantLock`, intrinsic locking offers no deterministic fairness guarantees, but the statistical fairness guarantees of most locking implementations are good enough for almost all situations. The language specification does not require the JVM to implement intrinsic locks fairly, and no production JVMs do. `ReentrantLock` does not depress lock fairness to new lows—it only makes explicit something that was present all along.

13.4. Choosing Between Synchronized and ReentrantLock

`ReentrantLock` provides the same locking and memory semantics as intrinsic locking, as well as additional features such as timed lock waits, interruptible lock waits, fairness, and the ability to implement non-block-structured locking. The performance of `ReentrantLock` appears to dominate that of intrinsic locking, winning slightly on Java 6 and dramatically on Java 5.0. So why not deprecate `synchronized` and encourage all new concurrent code to use `ReentrantLock`? Some authors have in fact suggested this, treating `synchronized` as a “legacy” construct. But this is taking a good thing *way* too far.

Intrinsic locks still have significant advantages over explicit locks. The notation is familiar and compact, and many existing programs already use intrinsic locking—and mixing the two could be confusing and error-prone. `ReentrantLock` is definitely a more dangerous tool than synchronization; if you forget to wrap the `unlock` call in a `finally` block, your code will probably appear to run properly, but you've created a time bomb that may well hurt innocent bystanders. Save `ReentrantLock` for situations in which you need something `ReentrantLock` provides that intrinsic locking doesn't.

`ReentrantLock` is an advanced tool for situations where intrinsic locking is not practical. Use it if you need its advanced features: timed, polled, or interruptible lock acquisition, fair queueing, or non-block-structured locking. Otherwise, prefer `synchronized`.

Under Java 5.0, intrinsic locking has another advantage over `ReentrantLock`: thread dumps show which call frames acquired which locks and can detect and identify deadlocked threads. The JVM knows nothing about which threads hold `ReentrantLock`s and therefore cannot help in debugging threading problems using `ReentrantLock`. This disparity is addressed in Java 6 by providing a management and monitoring interface with which locks can register, enabling locking information for `ReentrantLock`s to appear in thread dumps and through other management and debugging interfaces. The availability of this information

for debugging is a substantial, if mostly temporary, advantage for `synchronized`; locking information in thread dumps has saved many programmers from utter consternation. The non-block-structured nature of `ReentrantLock` still means that lock acquisitions cannot be tied to specific stack frames, as they can with intrinsic locks.

Future performance improvements are likely to favor `synchronized` over `ReentrantLock`. Because `synchronized` is built into the JVM, it can perform optimizations such as lock elision for thread-confined lock objects and lock coarsening to eliminate synchronization with intrinsic locks (see [Section 11.3.2](#)); doing this with library-based locks seems far less likely. Unless you are deploying on Java 5.0 for the foreseeable future and you have a *demonstrated* need for `ReentrantLock`'s scalability benefits on that platform, it is not a good idea to choose `ReentrantLock` over `synchronized` for performance reasons.

13.5. Read-write Locks

`ReentrantLock` implements a standard mutual-exclusion lock: at most one thread at a time can hold a `ReentrantLock`. But mutual exclusion is frequently a stronger locking discipline than needed to preserve data integrity, and thus limits concurrency more than necessary. Mutual exclusion is a conservative locking strategy that prevents writer/writer and writer/reader overlap, but also prevents reader/reader overlap. In many cases, data structures are “read-mostly”—they are mutable and are sometimes modified, but most accesses involve only reading. In these cases, it would be nice to relax the locking requirements to allow multiple readers to access the data structure at once. As long as each thread is guaranteed an up-to-date view of the data and no other thread modifies the data while the readers are viewing it, there will be no problems. This is what read-write locks allow: a resource can be accessed by multiple readers or a single writer at a time, but not both.

`ReadWriteLock`, shown in [Listing 13.6](#), exposes two `Lock` objects—one for reading and one for writing. To read data guarded by a `ReadWriteLock` you must first acquire the read lock, and to modify data guarded by a `ReadWriteLock` you must first acquire the write lock.

While there may appear to be two separate locks, the read lock and write lock are simply different views of an integrated read-write lock object.

Listing 13.6. ReadWriteLock Interface.

```
public interface ReadWriteLock {  
    Lock readLock();  
    Lock writeLock();  
}
```

The locking strategy implemented by read-write locks allows multiple simultaneous readers but only a single writer. Like `Lock`, `ReadWriteLock` admits multiple implementations that can vary in performance, scheduling guarantees, acquisition preference, fairness, or locking semantics.

Read-write locks are a performance optimization designed to allow greater concurrency in certain situations. In practice, read-write locks can improve performance for frequently accessed read-mostly data structures on multiprocessor systems; under other conditions they perform slightly worse than exclusive locks due to their greater complexity. Whether they are an improvement in any given situation is best determined via profiling; because `ReadWriteLock` uses `Lock` for the read and write portions of the lock, it is relatively easy to swap out a readwrite lock for an exclusive one if profiling determines that a read-write lock is not a win.

The interaction between the read and write locks allows for a number of possible implementations. Some of the implementation options for a `ReadWriteLock` are:

Release preference. When a writer releases the write lock and both readers and writers are queued up, who should be given preference—readers, writers, or whoever asked first?

Reader barging. If the lock is held by readers but there are waiting writers, should newly arriving readers be granted immediate access, or should they wait behind the writers? Allowing readers to barge ahead of writers enhances concurrency but runs the risk of starving writers.

Reentrancy. Are the read and write locks reentrant?

Downgrading. If a thread holds the write lock, can it acquire the read lock without releasing the write lock? This would let a writer “down-grade” to a read lock without letting other writers modify the guarded resource in the meantime.

Upgrading. Can a read lock be upgraded to a write lock in preference to other waiting readers or writers? Most read-write lock implementations do not support upgrading, because without an explicit upgrade operation it is deadlock-prone. (If two readers simultaneously attempt to upgrade to a write lock, neither will release the read lock.)

`ReentrantReadWriteLock` provides reentrant locking semantics for both locks. Like `ReentrantLock`, a `ReentrantReadWriteLock` can be constructed as nonfair (the default) or fair. With a fair lock, preference is given to the thread that has been waiting the longest; if the lock is held by readers and a thread requests the write lock, no more readers are allowed to acquire the read lock until the writer has been serviced and releases the write lock. With a nonfair lock, the order in which threads are granted access is unspecified. Downgrading from writer to reader is permitted; upgrading from reader to writer is not (attempting to do so results in deadlock).

Like `ReentrantLock`, the write lock in `ReentrantReadWriteLock` has a unique owner and can be released only by the thread that acquired it. In Java 5.0, the read lock behaves more like a `Semaphore` than a lock, maintaining only the count of active readers, not their identities. This behavior was changed in Java 6 to keep track also of which threads have been granted the read lock. ^[6]

^[6] One reason for this change is that under Java 5.0, the lock implementation cannot distinguish between a thread requesting the read lock for the first time and a reentrant lock request, which would make fair read-write locks deadlock-prone.

Read-write locks can improve concurrency when locks are typically held for a moderately long time and most operations do not modify the guard-

ed resources. `ReadWriteMap` in [Listing 13.7](#) uses a `ReentrantReadWriteLock` to wrap a `Map` so that it can be shared safely by multiple readers and still prevent reader-writer or writer-writer conflicts.^[7] In reality, `ConcurrentHashMap`'s performance is so good that you would probably use it rather than this approach if all you needed was a concurrent hash-based map, but this technique would be useful if you want to provide more concurrent access to an alternate `Map` implementation such as `LinkedHashMap`.

^[7] `ReadWriteMap` does not implement `Map` because implementing the view methods such as `entrySet` and `values` would be difficult and the “easy” methods are usually sufficient.

Listing 13.7. Wrapping a `Map` with a Read-write Lock.

```
public class ReadWriteMap<K,V> {
    private final Map<K,V> map;
    private final ReadWriteLock lock = new ReentrantReadWriteLock();
    private final Lock r = lock.readLock();
    private final Lock w = lock.writeLock();

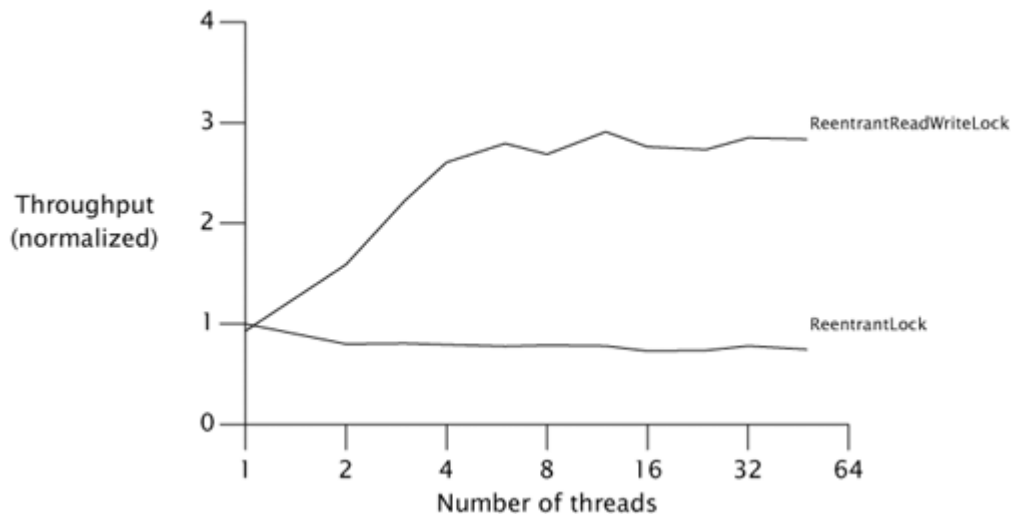
    public ReadWriteMap(Map<K,V> map) {
        this.map = map;
    }

    public V put(K key, V value) {
        w.lock();
        try {
            return map.put(key, value);
        } finally {
            w.unlock();
        }
    }
    // Do the same for remove(), putAll(), clear()

    public V get(Object key) {
        r.lock();
        try {
            return map.get(key);
        } finally {
            r.unlock();
        }
    }
    // Do the same for other read-only Map methods
}
```

Figure 13.3 shows a throughput comparison between an `ArrayList` wrapped with a `ReentrantLock` and with a `ReadWriteLock` on a four-way Opteron system running Solaris. The test program used here is similar to the `Map` performance test we've been using throughout the book—each operation randomly selects a value and searches for it in the collection, and a small percentage of operations modify the contents of the collection.

Figure 13.3. Read-write Lock Performance.



Summary

Explicit `Lock`s offer an extended feature set compared to intrinsic locking, including greater flexibility in dealing with lock unavailability and greater control over queueing behavior. But `ReentrantLock` is not a blanket substitute for `synchronized`; use it only when you need features that `synchronized` lacks.

Read-write locks allow multiple readers to access a guarded object concurrently, offering the potential for improved scalability when accessing read-mostly data structures.